

# PES University

Department of Computer Science and Engineering

## Blockchain-Based Charity Donation Platform

*A Decentralized Application for Transparent Charitable Giving*

**Subject Code: UE23CS342BA5**

**Subject Name: Block Chain**

### Team Details

| Name                 | SRN           | Section  |
|----------------------|---------------|----------|
| Saiprasanna          | PES2UG23AM090 | AIML - B |
| Sudarshan Srinivasan | PES2UG23AM106 | AIML - B |

Academic Year: 2025-26 | Semester: VI

GitHub: [https://github.com/ProSai45/Charity\\_Blockchain](https://github.com/ProSai45/Charity_Blockchain)

# Table of Contents

|                                             |    |
|---------------------------------------------|----|
| 1. Introduction                             | 4  |
| 2. Problem Statement                        | 5  |
| 3. Objectives                               | 6  |
| 4. Functional Requirements                  | 7  |
| 4.1 Smart Contract Requirements             | 7  |
| 4.2 Frontend Requirements                   | 8  |
| 5. Type of Blockchain                       | 9  |
| 5.1 Classification                          | 9  |
| 5.2 Why Ethereum-Compatible?                | 10 |
| 6. Consensus Algorithm — Justification      | 11 |
| 6.1 Ganache: Instant Mining                 | 11 |
| 6.2 Production Target: Proof of Stake (PoS) | 12 |
| 6.3 Why PoS is Appropriate                  | 13 |
| 7. Technology Stack                         | 14 |
| 8. Smart Contract Design                    | 15 |
| 8.1 Contract Overview                       | 15 |
| 8.2 Data Structures                         | 16 |
| 8.3 Access Control Modifiers                | 17 |
| 8.4 All Functions                           | 18 |
| 8.5 Events                                  | 20 |
| 9. Implementation Details                   | 22 |
| 9.1 Network Setup                           | 22 |
| 9.2 Smart Contract Deployment               | 23 |
| 9.3 API Integration                         | 24 |
| 9.4 UI Implementation                       | 25 |
| 10. Testing and Validation                  | 27 |
| 10.1 Test Suite Overview                    | 27 |
| 10.2 Test Categories                        | 28 |
| 10.3 Validation Results                     | 29 |
| 11. Results and Discussion                  | 30 |
| 11.1 Campaign Registration                  | 30 |
| 11.2 Verifier Approval Workflow             | 31 |
| 11.3 Donation with Impact Receipt           | 32 |
| 11.4 Donor Dashboard                        | 33 |
| 11.5 Proof Update Trail                     | 34 |
| 12. Security Analysis                       | 35 |
| 12.1 Reentrancy Attack Prevention           | 35 |
| 12.2 ETH Transfer Safety                    | 36 |
| 12.3 Role-Based Access Control              | 37 |
| 12.4 Input Validation                       | 38 |
| 12.5 Zero Custody Model                     | 39 |
| 12.6 Immutability Guarantees                | 40 |
| 13. GitHub Repository Link                  | 41 |
| 14. Conclusion                              | 42 |
| 15. References                              | 43 |

# 1. Introduction

---

The rollout of blockchain technology has completely changed the way we've done business by creating transparent, immutable, and trustless transactions in many different industries. A blockchain-based charity donation platform was created as a decentralized application (DApp) built on the Ethereum blockchain to address the chronic lack of accountability and transparency that now exists within the traditional charitable giving framework.

In traditional charity donation systems, donors send money to an organization that then sends that money out via a 3rd party intermediary, leaving the donor without visibility into how their funds are used. This lack of visibility creates distrust between the donor and the charity, thus reducing the overall effectiveness of the charitable campaign and the amount of money that could be raised from the donor.

Using the Blockchain donation platform, there will be complete transparency and traceability of every rupee keystroked into the wallet of a donor, from the time it is sent to the verified beneficiary.

The platform uses Solidity Smart Contracts running on a Ganache blockchain (a local private Ethereum blockchain), React (a JavaScript library for building user interfaces), MetaMask (a wallet used to connect to the public Ethereum network), and ethers.js (a JavaScript library for communicating with the Ethereum Blockchain). All of the above-mentioned items are combined together to create an immutable and permanent on-chain record of every campaign, donation, verification decision, and proof update.

# 2. Problem Statement

---

Many traditional charity organizations have popular issues that damage donors confidence and limit their impact, like the following.

- No transparency - Donors cannot track where their donations were used after they've donated.
- Intermediary risk - Funds go through numerous intermediaries which opens up the donation to mismanagement or fraud.
- No accountability monitoring system - There is not a uniform way to verify that the charitable donation items or funds actually made it to the intended beneficiary.
- Donor concerns about legitimacy - There isn't a reliable way for a donor to determine whether a charitable donation or charitable organization is legitimate prior to contributing.
- Irreversibility of the donation with no traceability - Once a donation is made, the donor loses contact with regard to the transaction even though it is for all time.

The main issue is that there is no immutable, trustless, or transparent system to track and trace the entire donation history, from donor to beneficiary, fraud prevention features included.

### 3. Objectives

---

Write a smart contract in Solidity that records and tracks donor contributions (ETH) to recipients without going through a third party, by allowing donations to be sent directly from the donor's wallet to the recipient's wallet with one transaction.

Create a tiered verification system using trusted verifiers who will check the legitimacy of donation campaigns before accepting donations. This system will reduce fraudulent donation campaigns.

Create an immutable proof trail that shows the use of donations, delivery of donations, and completion of donations, by providing donors with proof of how their donation was used, at any point in time.

Implement a role-based access control system that has four major user roles: Owner (the person responsible for managing the platform and contract), Verifier (a trusted auditor who verifies the legitimacy of each donation), Creator (the charity that creates a donation), Donor (the person who donates). Each role will have a clearly defined set of permissions.

Build a web-based front-end user interface using React, that will allow donors to browse donation campaigns, donate to these campaigns, and view their donation status in real time through a web3 interface.

Develop a complete campaign lifecycle management system which includes: creating campaigns, approving campaigns, pausing campaigns, reactivating campaigns and closing campaigns, with a complete audit trail.

Complete a comprehensive set of testing on the local Ganache network to validate all edge cases through 46 different test cases.

## 4. Functional Requirements

---

### 4.1 Smart Contract Requirements

- Any Ethereum address can create a charity campaign via the contract, specifying the following: Title, Description Hash, Category, Beneficiary Wallet, Target Amount and a Proof Bundle Hash (Optional).
- When created, the campaign will have a (PENDING\_VERIFICATION) status meaning that it cannot receive donations until a verifier or the owner has approved it.
- Donations will send ETH from the donor's wallet directly to the beneficiary's wallet — all in one transaction. At no time will the contract hold any funds.
- The deployer of the contract (the owner) is the only entity that can add or remove verifiers.
- Verifiers and the owner will both have the ability to approve campaigns (with a verification tier), reject campaigns, pause campaigns that are currently active, and resume campaigns that have been paused.
- The creator of the campaign, the owner and verifiers will be able to create records for the Proof updates and the records will be permanently stored on-chain.
- For all state changes, there will be an event emitted which can be read by the frontend of the application.

## 4.2 Frontend Requirements

- The wallet connection status of MetaMask will automatically update to show the status of your wallet, your currently connected account, and the network you're currently using.
- The campaigns section of the website will show all live campaigns that were retrieved from the blockchain as well as support for filtering through the campaigns by their current state (active/inactive).
- The detail screen for an individual campaign will display the current level of funds raised towards the goal amount; a list of all supporters; a timeline for updates on how the donation has been used; and a donation form to make your purchase (if you are making a donation).
- The donor's dashboard will show the full donation history of the connected wallet.
- The verifier's panel will show the options available for approving, denying, pausing, and updating proof for a campaign. Only individuals that are connected with the appropriate user role will be able to use these functions.
- The current state of all transactions (waiting to be signed; submitted; confirmed; and failed) will appear within your user interface as soon as the transaction becomes accessible to other users.

## 5. Type of Blockchain

---

### 5.1 Classification

This project uses a Private/Local Permissioned Blockchain for development and demonstration purposes, implemented through Ganache — a personal Ethereum blockchain simulator.

| Property          | This Project             | Public Ethereum       |
|-------------------|--------------------------|-----------------------|
| Network type      | Local private (Ganache)  | Public permissionless |
| Participants      | Controlled test accounts | Anyone globally       |
| Consensus         | Instant mining           | Proof of Stake (PoS)  |
| Transaction speed | Instant (< 1 second)     | ~12 seconds per block |
| Cost              | Zero (test ETH)          | Real ETH gas fees     |
| Transparency      | Local only               | Global public ledger  |
| Use case          | Development & demo       | Production deployment |

### 5.2 Why Ethereum-Compatible?

Ethereum was selected as the target blockchain because it is the first to introduce smart contracts (a type of self-executing code that runs on the blockchain), has the largest developer community, provides access to a well-documented programming language called Solidity, and

uses industry-standard tools such as Truffle, Ganache and MetaMask. Additionally, the application that the project creates will be able to run on all Ethereum-compatible networks (for example Sepolia Testnet and Polygon) by simply modifying the network configuration in `truffle-config.js`.

## 6. Consensus Algorithm — Justification

---

### 6.1 Ganache: Instant Mining (Development Consensus)

Ganache implements instant mining in the local development environment; each submitted transaction is put into a new block as soon as they are submitted, without any competition for validation. This mechanism is not a genuine consensus algorithm but rather a development simulation mode that allows fast transaction processing.

- All submitted transactions will be immediately added with less than 100 milliseconds.
- There is no mining competition, no concept of nonces or validators.
- The blocks are timestamped with actual system times.
- The chain ID is set to 1337, which is used by MetaMask to identify this network as a local chain.

### 6.2 Production Target: Proof of Stake (PoS)

Ethereum's production consensus mechanism since The Merge in September 2022 is Proof of Stake (PoS). Under PoS:

- To become a validator, it is required to stake at least 32 ETH as collateral to confirm the block.
- Validators are randomly selected to publish blocks based on the amount they have been staked.
- The validator will send an attest (vote) to the published block, and if 2/3 of all validators send an attest, then the block is ordered.
- Any malicious validator that participates in dishonest behavior will lose their staked amount through a slashing and be economically deterred from continuing to participate.
- Energy consumption goes down about ===='Proof of Work" by more than 99.95% compared to Proof of Stake.

### 6.3 Why PoS is Appropriate for This Platform

A charity donation platform demands high accountability and low cost. PoS on Ethereum provides:

- Finality: Once a transaction has been completed within two epochs (~12 minutes), it can no longer be altered; therefore, any record of donations cannot be tampered with.
- Energy Efficient: This application is well suited to deliver social good with minimal energy used.
- Lower gas Fees: Compared to PoW protocols, gas is cheaper so small donations are feasible.
- Decentralized: No individual has control over the ability to validate transactions - this directly supports and aligns with the goal of providing a trustless donation platform.

## 7. Technology Stack

| Layer             | Technology           | Version      | Purpose                                          |
|-------------------|----------------------|--------------|--------------------------------------------------|
| Smart Contract    | Solidity             | 0.8.20       | Contract logic, storage, events, access control  |
| Smart Contract    | OpenZeppelin         | ^5.0         | Ownable, ReentrancyGuard security primitives     |
| Local Blockchain  | Ganache              | v7.9.2       | Local Ethereum simulator, Chain ID 1337          |
| Build & Deploy    | Truffle              | Latest       | Compile, migrate, test smart contracts           |
| Frontend          | React                | ^18.3.1      | UI component framework                           |
| Frontend          | Vite                 | ^5.4.10      | Fast dev server, build tool, ES module bundler   |
| Blockchain Bridge | ethers.js            | ^6.13.2      | Provider, signer, Contract, ABI encoding         |
| Routing           | React Router DOM     | ^6.28.1      | Client-side routing for multi-page DApp          |
| Wallet            | MetaMask             | Browser ext. | Key management, transaction signing, web3 bridge |
| Testing           | Truffle Test + Mocha | 46 tests     | Smart contract unit & integration testing        |
| Runtime           | Node.js              | 20.x         | JavaScript runtime for tooling                   |

## 8. Smart Contract Design

### 8.1 Contract Overview

CharityDonationPlatform.sol provides all of the backend functionality for the entire application. It uses OpenZeppelin's Ownable and ReentrancyGuard as its base. It contains three enumerations, three user defined data types (structs), six mappings, four modifiers, fourteen method implementations, nine event declarations, and thirteen custom error declarations.

### 8.2 Data Structures

#### Enums

| Enum           | Values                                                    | Purpose                                  |
|----------------|-----------------------------------------------------------|------------------------------------------|
| CampaignStatus | PENDING_VERIFICATION, ACTIVE, PAUSED, COMPLETED, REJECTED | Tracks the lifecycle state of a campaign |

| Enum             | Values                                                                            | Purpose                                        |
|------------------|-----------------------------------------------------------------------------------|------------------------------------------------|
| VerificationTier | UNVERIFIED, SELF_DECLARED, NGO_VERIFIED, ADMIN_VERIFIED                           | Indicates how thoroughly a campaign was vetted |
| UpdateType       | GENERAL_UPDATE, PROOF_OF_USE, DELIVERY_CONFIRMATION, STATUS_CHANGE, CLOSURE_PROOF | Classifies the type of proof update posted     |

## Structs

Struct for Campaign: Has the following variables - id, creator address, beneficiaryWallet, title, descriptionHash, category, targetAmount, raisedAmount, donationCount, CampaignStatus, VerificationTier, createdAt, verifiedAt, closedAt, and latestProofHash.

Struct for Donation: Contains the following variables - id, campaignId, donor address, beneficiaryWallet, amount (in wei), and timestamp.

Struct for CampaignUpdate: Includes the following variables - id, campaignId, author address, UpdateType, detailsHash (bytes32), and timestamp.

## Storage Mappings

| Mapping             | Key → Value                | Purpose                                             |
|---------------------|----------------------------|-----------------------------------------------------|
| campaigns           | uint256 → Campaign         | Look up any campaign by its sequential ID           |
| donationsByCampaign | uint256 → Donation[]       | All donations for a specific campaign               |
| donationsById       | uint256 → Donation         | Direct lookup of a single donation by its ID        |
| donationIdsByDonor  | address → uint256[]        | All donation IDs ever made by a wallet address      |
| updatesByCampaign   | uint256 → CampaignUpdate[] | Full proof update history for a campaign            |
| verifiers           | address → bool             | O(1) check whether an address is a trusted verifier |

## 8.3 Access Control Modifiers

| Modifier                     | Who Passes                                | Applied To                                                     |
|------------------------------|-------------------------------------------|----------------------------------------------------------------|
| onlyOwner (OZ)               | Contract deployer (Account 0) only        | setVerifier()                                                  |
| onlyOwnerOrVerifier          | Owner OR any address in verifiers mapping | approveCampaign, rejectCampaign, pauseCampaign, resumeCampaign |
| onlyCreatorOrOwnerOrVerifier | Campaign's creator, owner, or verifier    | addCampaignUpdate, completeCampaign                            |



| Modifier          | Who Passes                               | Applied To       |
|-------------------|------------------------------------------|------------------|
| nonReentrant (OZ) | Blocks re-entrant calls during execution | donateToCampaign |

## 8.4 All Functions

### createCampaign()

Visibility: external | Access: any address | Gas: write

Parameters: title (string calldata), descriptionHash (string calldata), category (string calldata), beneficiaryWallet (address), targetAmount (uint256), proofBundleHash (bytes32)

Use custom error messages (e.g., Empty Title, Empty Description Hash, Empty Category) to validate the five required parameters (title, descriptionHash, category, Beneficiary Wallet, Target Amount). Assign a sequential campaignId to the CampaignMapping based upon the nextCampaignId counter. Create a Campaign Mapping with CampaignId = campaignId; Status = PENDING\_VERIFICATION; Verification Tier = UNVERIFIED; CreatedAt = block.timestamp. Emit the Campaign Created event

```
campaigns[campaignId] = Campaign({ id: campaignId, creator: msg.sender,
beneficiaryWallet: beneficiaryWallet, status:
CampaignStatus.PENDING_VERIFICATION, verificationTier:
VerificationTier.UNVERIFIED, createdAt: block.timestamp, ...});
```

### setVerifier(address verifier, bool allowed)

Visibility: external | Access: onlyOwner | Gas: write

Sets verifiers[verifier] = allowed. Pass true to add a verifier, false to remove one. Emits VerifierUpdated event. Reverts with Unauthorized if verifier is the zero address.

### approveCampaign(uint256 campaignId, VerificationTier tier)

Visibility: external | Access: onlyOwnerOrVerifier | Gas: write

Checks campaign is in PENDING\_VERIFICATION or PAUSED status (to allow re-approval after a pause). Rejects if tier is UNVERIFIED. Sets status=ACTIVE, verificationTier=tier, verifiedAt=block.timestamp. Emits CampaignApproved.

### rejectCampaign(uint256 campaignId, bytes32 reasonHash)

Visibility: external | Access: onlyOwnerOrVerifier | Gas: write

Campaign must be in PENDING\_VERIFICATION or PAUSED status. Sets status=REJECTED. The reasonHash is a bytes32 keccak256 hash of the rejection reason document stored off-chain. Emits CampaignRejected.

### pauseCampaign(uint256 campaignId, bytes32 reasonHash)

Visibility: external | Access: onlyOwnerOrVerifier | Gas: write

Campaign must be ACTIVE. Sets status=PAUSED, halting new donations immediately. Emits CampaignPaused with a reason hash. Can be resumed or permanently rejected afterward.

**resumeCampaign(uint256 campaignId)**

Visibility: external | Access: onlyOwnerOrVerifier | Gas: write

Campaign must be PAUSED. Sets status=ACTIVE. Emits CampaignResumed. Donations can flow again immediately after resumption.

**donateToCampaign(uint256 campaignId) payable nonReentrant**

Visibility: external payable | Access: any address | Gas: write + ETH transfer

This is the most critical function. Checks: (1) campaign.status == ACTIVE, (2) msg.value > 0, (3) beneficiaryWallet != address(0). Creates a Donation struct stored in all three donation mappings. Increments campaign.raisedAmount and donationCount. Transfers ETH directly to beneficiaryWallet using the low-level .call pattern. Checks the return value and reverts with TransferFailed if the transfer fails. Emits DonationRecorded.

```
(bool success, ) = payable(campaign.beneficiaryWallet).call{value:
msg.value}("");if (!success) revert TransferFailed(campaign.beneficiaryWallet,
msg.value);// ETH leaves in the SAME transaction. Contract balance stays at zero.
```

**addCampaignUpdate(uint256 campaignId, UpdateType updateType, bytes32 detailsHash)**

Visibility: external | Access: onlyCreatorOrOwnerOrVerifier | Gas: write

Validates detailsHash != bytes32(0). Blocks updates on COMPLETED or REJECTED campaigns. Creates a CampaignUpdate struct pushed to updatesByCampaign[campaignId]. Also updates campaign.latestProofHash to the new hash. Emits CampaignUpdateAdded. This function is the core of the immutable proof trail.

**completeCampaign(uint256 campaignId)**

Visibility: external | Access: onlyCreatorOrOwnerOrVerifier | Gas: write

Reverts if campaign is already COMPLETED or REJECTED. Sets status=COMPLETED and closedAt=block.timestamp. Emits CampaignCompleted. No further donations or updates are accepted after this point.

**getCampaign(uint256 campaignId) view**

Returns the full Campaign memory struct. Reverts with CampaignNotFound if the campaign does not exist (campaigns[id].id == 0 check).

**getCampaignCount() view**

Returns nextCampaignId - 1 — the total number of campaigns created. Used by the frontend to loop from 1 to count fetching each campaign.

**getCampaignDonations(uint256 campaignId) view**

Returns the full Donation[] array for a campaign from donationsByCampaign mapping.

**getCampaignUpdates(uint256 campaignId) view**

Returns the full CampaignUpdate[] array from updatesByCampaign mapping — the complete immutable proof trail for a campaign.

**getDonationsByDonor(address donor) view**

Iterates donationIdsByDonor[donor] (array of IDs) and fetches each Donation from donationsById mapping. Returns the complete donation history for any wallet address. Used by the Donor Dashboard page.

**8.5 Events**

| Event               | Parameters                                                                             | Emitted When                  |
|---------------------|----------------------------------------------------------------------------------------|-------------------------------|
| CampaignCreated     | campaignId (indexed), creator (indexed), beneficiaryWallet (indexed), title            | New campaign registered       |
| CampaignApproved    | campaignId (indexed), verificationTier (uint8)                                         | Verifier approves campaign    |
| CampaignRejected    | campaignId (indexed), reasonHash                                                       | Campaign rejected by verifier |
| DonationRecorded    | donationId (indexed), campaignId (indexed), donor (indexed), beneficiaryWallet, amount | ETH donation confirmed        |
| CampaignUpdateAdded | campaignId (indexed), updateId (indexed), updateType, detailsHash                      | Proof update posted           |
| CampaignPaused      | campaignId (indexed), reasonHash                                                       | Active campaign paused        |
| CampaignResumed     | campaignId (indexed)                                                                   | Paused campaign resumed       |
| CampaignCompleted   | campaignId (indexed)                                                                   | Campaign marked complete      |
| VerifierUpdated     | verifier (indexed), allowed (bool)                                                     | Verifier added or removed     |

**9. Implementation Details****9.1 Network Setup**

The project runs on a local Ganache blockchain with the following configuration:

| Parameter       | Value                                                  |
|-----------------|--------------------------------------------------------|
| RPC URL         | http://127.0.0.1:8545                                  |
| Chain ID        | 1337                                                   |
| Network ID      | 1337                                                   |
| Mnemonic        | test test test test test test test test test test junk |
| Accounts        | 30 deterministic accounts, each with 1000 ETH          |
| Block gas limit | 30,000,000                                             |
| Mining mode     | Instant (transaction triggers block)                   |

Ganache startup command:

```
ganache --wallet.mnemonic "test test...junk" --wallet.totalAccounts 30
--wallet.defaultBalance 1000 --chain.chainId 1337 --chain.networkId 1337
--server.host 127.0.0.1 --server.port 8545
```

## 9.2 Smart Contract Deployment

Truffle handles compilation and deployment. The configuration in truffle-config.js specifies:

- Solidity compiler version: 0.8.20 (local solc package for offline reproducibility)
- Optimizer: enabled, 200 runs
- Network: development on 127.0.0.1:8545, Chain ID 1337

Deployment sequence:

```
npm run compile      # Compiles .sol → ABI + bytecode in build/contracts/npm test
# Validates 46 test cases on fresh Ganache instance
npm run migrate:reset # Deploys to running Ganache, writes address to artifact JSON
```

After migration, Truffle writes the deployed contract address into build/contracts/CharityDonationPlatform.json under the networks object. The frontend's contractConfig.js reads this file automatically — no manual address copying required.

## 9.3 API Integration (ethers.js ↔ Contract)

The frontend communicates with the smart contract exclusively through ethers.js v6. The integration layer has four tiers:

### Tier 1: contractConfig.js — Single source of truth

```
export const CONTRACT_ADDRESS = import.meta.env.VITE_CONTRACT_ADDRESS ||
getLatestArtifactAddress(); // auto-reads from Truffle artifact
export const contractConfig = { abi, address, supportedChainId: 1337 };
```

### Tier 2: getPlatformContract(runner) — Contract instance factory

```
// For READ operations (no MetaMask popup)
const contract = getPlatformContract(provider); // provider = BrowserProvider
await contract.getCampaign(1); // returns raw BigInt struct
// For WRITE operations (triggers MetaMask popup)
const signer = await provider.getSigner();
const contract = getPlatformContract(signer);
await contract.donateToCampaign(1, { value: parseEther("0.1") });
```

### Tier 3: WithSigner functions — All write operations

Every state-changing operation has a dedicated function: createCampaignWithSigner, donateToCampaignWithSigner, approveCampaignWithSigner, rejectCampaignWithSigner, pauseCampaignWithSigner, resumeCampaignWithSigner, addCampaignUpdateWithSigner, completeCampaignWithSigner, setVerifierWithSigner.

### Tier 4: normalize functions — BigInt → JS Number conversion

The EVM encodes all uint256 values as BigInt in ethers.js v6. normalizeCampaign(), normalizeDonation(), and normalizeCampaignUpdate() convert BigInt IDs and counts to

Number while preserving raisedAmount and targetAmount as BigInt for accurate formatEther() conversion.

## 9.4 UI Implementation

The React frontend is organized into a clean layered architecture:

| Layer      | Files                                                                                                                                     | Responsibility                                                         |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| Entry      | main.jsx                                                                                                                                  | Mounts React root, wraps with BrowserRouter                            |
| Router     | App.jsx                                                                                                                                   | Defines all routes, nests pages inside AppShell                        |
| Shell      | AppShell.jsx                                                                                                                              | Calls useWallet() once, passes wallet via Outlet context to all pages  |
| Hooks      | useWallet, useCampaigns, useCampaignDetail, useDonorDashboard, usePlatformOverview                                                        | Blockchain data fetching with React state, loading, and error handling |
| Utils      | campaigns.js, contractConfig.js, formatters.js, chain.js                                                                                  | Raw blockchain calls, validation, data formatting                      |
| Pages      | HomePage, CampaignsPage, CampaignDetailPage, DonorDashboardPage, VerifierPanelPage                                                        | Page layouts, combine hooks and components                             |
| Components | WalletConnector, CreateCampaignForm, DonationForm, CampaignCard, CampaignAdminActions, VerifierManagementForm, NetworkBanner, StatusBadge | Reusable UI pieces, some with direct blockchain write calls            |

The wallet state pattern allows you to get your wallet from the app shell via the useWallet() call and then use React Router's outlet context to provide it to the Outlet's children. This means that every child page will receive the wallet state by calling the useOutletContext() hook and that you will always have only one wallet state across the entire application, and the state will automatically update reactively to any change the user makes, such as switching networks or accounts using MetaMask.

## 10. Testing and Validation

## 10.1 Test Suite Overview

The project includes 46 automated Truffle tests written in JavaScript using Mocha and Chai. Tests run against a fresh Ganache instance on each execution, guaranteeing a clean state.

```
npm test → 46 passing (~15-30 seconds)
```

## 10.2 Test Categories

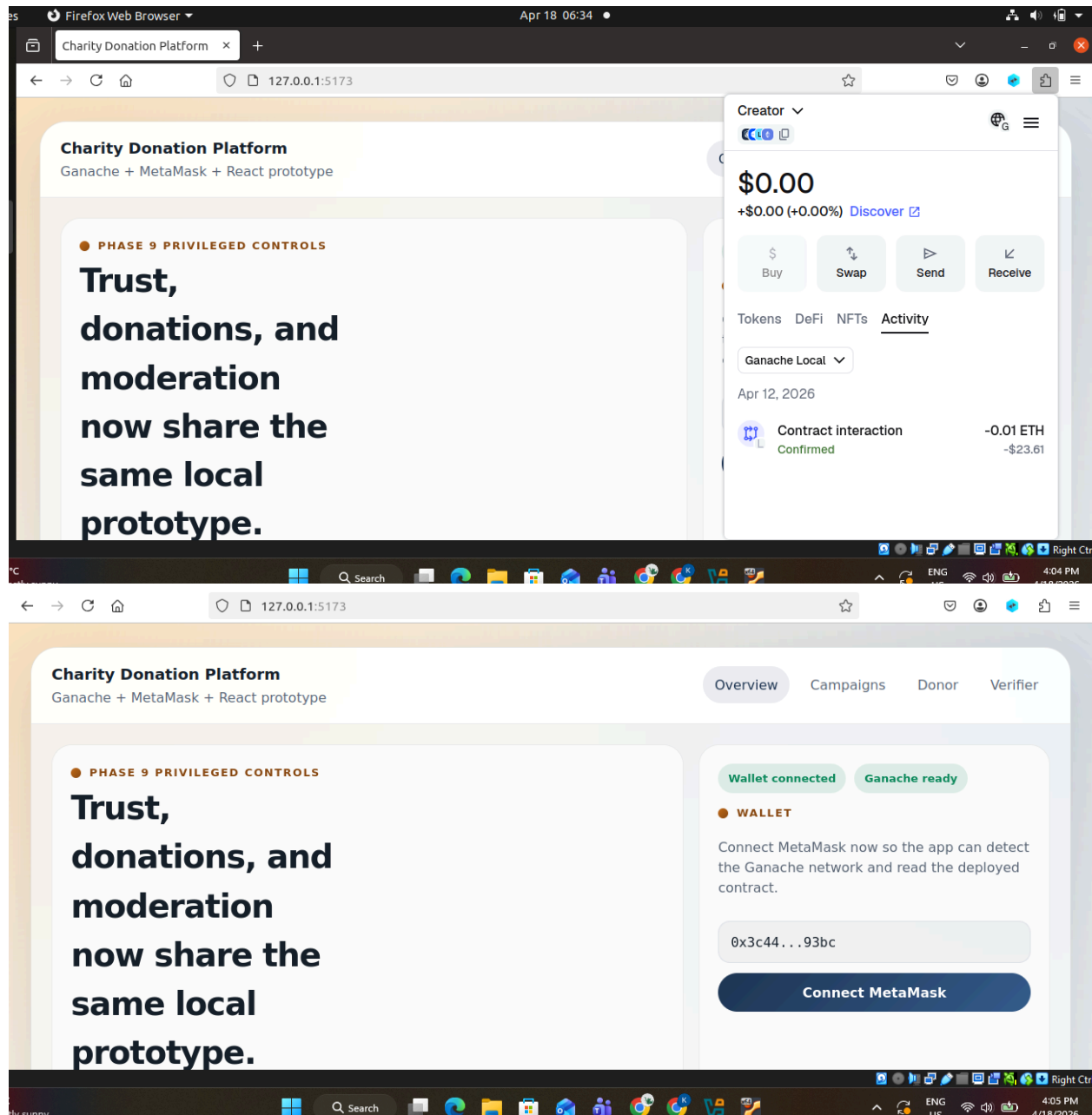
| Category                   | Tests Covered                                                             | Key Assertions                                                |
|----------------------------|---------------------------------------------------------------------------|---------------------------------------------------------------|
| Campaign Creation          | Valid creation, empty field rejections, sequential ID assignment          | CampaignCreated event, PENDING_VERIFICATION status            |
| Verifier Management        | Add/remove verifier, non-owner rejection                                  | verifiers mapping updated, VerifierUpdated event              |
| Campaign Approval          | Approval with all tiers, invalid tier rejection, wrong status rejection   | ACTIVE status, verifiedAt timestamp, tier stored correctly    |
| Campaign Rejection & Pause | Reject pending, pause active, resume paused                               | Status transitions, events emitted                            |
| Donation Flow              | Successful donation, zero donation rejection, inactive campaign rejection | DonationRecorded event, raisedAmount updated, ETH transferred |
| Proof Updates              | Valid update, zero hash rejection, completed campaign rejection           | latestProofHash updated, update stored in array               |
| Campaign Completion        | Creator/verifier/owner can complete, double-complete rejection            | COMPLETED status, closedAt timestamp, CampaignCompleted event |
| Security                   | Reentrancy protection, unauthorized access rejections                     | Custom errors thrown, state unchanged after failed calls      |
| Read Functions             | getCampaignCount, getDonationsByDonor, getCampaignUpdates                 | Correct data returned, empty arrays for no data               |

## 10.3 Validation Results

All 46 tests pass on the local Ganache network. The uWS binary compatibility warning (Cannot find module uws\_linux\_x64\_127.node) is a known benign warning from the bundled Ganache version — it falls back to a Node.js implementation and all functionality is unaffected.

# 11. Results and Discussion

## 11.1 Campaign Registration



**● NETWORK STATUS**  
Connected to the expected Ganache network. Contract reads are enabled.

Chain ID 1337

**Platform readiness**  
This frontend now connects MetaMask, detects Ganache, reads contract overview data, and exposes live campaign browsing routes.

Contract address

0x6C2d83262fF84cBaDb3e416D527403135D757892

Connected account role

donor

Contract owner

0xf39F...2266

Latest refresh

6:33:26 AM

**Campaign count**  
Live contract read

0

This value comes from  
'getCampaignCount()' and confirms the  
React app can read the deployed  
Ganache contract.

Latest refresh

6:33:26 AM

Refresh contract read

**Frontend progress**  
Implemented through Phase 9

MetaMask connect

Available

Ganache network check

Ganache Local (1337)

Campaign list + detail routes

Live

Create campaign form

Live

**Next UI phases**  
What the current frontend foundation is preparing for

Phase 10 focuses on final polish, demo packaging, Ubuntu setup clarity, and the end-to-end reviewer flow.



### Campaign explorer

This Phase 7 view reads live campaigns from the deployed Ganache contract and exposes the PRD-defined creation inputs.

### Campaign listing and creation

Campaigns are loaded from `getCampaignCount()` plus `getCampaign(id)`. Creation writes directly through MetaMask using the same contract address resolved in Phase 6.

**Ganache connected**

**All** Pending Active Paused Completed Rejected

**Refresh campaigns**

No campaigns match the current filter yet. Create one below or switch filters.

### Create campaign

PRD-required input set

**Title**

**Description hash**

**Category** **Target amount (ETH)**

**Beneficiary wallet**

**Proof bundle hash (optional)**

**Create campaign**

**Reset form**

### Live summary

Quick trust and deployment context for the current frontend phase

|                       |                     |                     |                 |
|-----------------------|---------------------|---------------------|-----------------|
| <b>CAMPAIGN COUNT</b> | <b>ACTIVE TOTAL</b> | <b>TOTAL RAISED</b> | <b>CONTRACT</b> |
| 0                     | 0                   | 0 ETH               | 0x6C2d8326...   |

### Donor trust dashboard

Phase 8 adds donation tracking, donor impact receipts, and campaign support visibility.

|                      |                 |                            |                           |
|----------------------|-----------------|----------------------------|---------------------------|
| <b>TOTAL DONATED</b> | <b>RECEIPTS</b> | <b>SUPPORTED CAMPAIGNS</b> | <b>VERIFIED SUPPORTED</b> |
| 0 ETH                | 0               | 0                          | 0                         |

**Refresh donor history** **Browse campaigns**

### Donor impact receipts

Each entry is reconstructed from immutable on-chain donation history.

No donor receipts yet. Open a live campaign and make the first donation from this wallet.

### Connected donor

Current wallet context

**Wallet** 0x3c44cdddb6a900fa2b585dd299e03d12fa4293bc

**Network** **Ganache ready**

**Active supported** 0

### Support footprint

Quick trust-facing summary of the campaigns this wallet has backed.

Campaign support metrics will appear after the first donation.

the PRD-defined creation inputs.

## Campaign listing and creation

Campaigns are loaded from ``getCampaignCount()`` plus ``getCampaign(id)``. Creation writes directly through MetaMask using the same contract address resolved in Phase 6.

Ganache connected

All

Pending

Active

Paused

Completed

Rejected

Refresh campaigns

No campaigns match the current filter yet. Create one below or switch filters.

### Title

Food For Homeless People

### Description hash

Distributing the food everywhere

### Category

Food

### Target amount (ETH)

2

### Beneficiary wallet

6a900fa2b585dd299e03d12FA4293BQ

### Proof bundle hash (optional)

0x...

Create campaign

The screenshot displays the Charity Donation Platform interface. The top section, titled "Campaign listing and creation", explains that campaigns are loaded from `getCampaignCount()` and `getCampaign(id)`, and are created via MetaMask. It shows a "Ganache connected" status and filter buttons: All, Pending, Active, Paused, Completed, and Rejected. A "Refresh campaigns" button is present. Below this, a message states: "No campaigns match the current filter yet. Create one below or switch filters."

The bottom section shows a "Refresh campaigns" button and a list of campaigns. The first campaign, "CAMPAIGN #1 Food For Homeless People", is in a "Pending Verification" state. It lists details: Food, Raised (0 ETH), Target (2 ETH), Funding progress (0%), and Donations (0). To the right, a "Beneficiary wallet" field shows "0x...", and a "Proof bundle hash (optional)" field also shows "0x...". A "Create campaign" button is visible, followed by a "Reset form" button. A green notification states: "Campaign creation transaction mined successfully." Below this, the transaction hash is displayed: "0x72409c92e768afc903ae981fe3f09b7a1432d4079c1".

A MetaMask "Transaction request" popup is visible on the right side of the screen. It shows the following details:

- Network: Ganache Local
- Request from: HTTP 127.0.0.1:5173
- Interacting with: 0x6C2d8...57892
- Network fee: 0.0005 ETH (\$1.18)
- Speed: 0.0005 \$1.18
- Max fee: 0.0005 \$1.18
- Nonce: 47
- Data: Hex 0x3edba5e800000000000000000000000000000000

The MetaMask popup includes "Cancel" and "Confirm" buttons.

## 11.2 Verifier Approval Workflow

As soon as Account 1 (the one who approved or verified the campaign via Account 0) goes into the Verifier Panel screen and approves the campaign using the NGO\_VERIFIED tier, the state of the campaign becomes recorded as On-Chain (i.e., Active). An immutably recorded timestamp of when the campaign was approved (i.e., verified) as per Account 0 will also be created. Donors now will see the campaign as being included in the Active filter.

The screenshot displays the Verifier Panel interface. At the top, there are four status cards: 'CURRENT ROLE' (owner-admin), 'VERIFIER ACCESS' (Not granted), 'CAMPAIGNS LOADED' (1), and 'ACTIONABLE VIEW' (1). Below these are filters for 'Actionable', 'All', 'Pending', 'Active', 'Paused', 'Completed', and 'Rejected', along with a 'Refresh panel' button. On the right, there are toggle switches for 'Owner-admin' (Yes) and 'Verifier' (No), and summary statistics for 'Pending campaigns' (1) and 'Paused campaigns' (0). The 'Verifier management' section indicates that only the owner-admin has access to delegated reviewer accounts. The 'Verifier wallet' section shows a truncated address '0x...'. A Windows taskbar is visible at the bottom of the main window. Below this, a detailed view of the 'Transaction request' is shown, including fields for Network, Request from (HTTP), Interacting with (0x6), Method, Network fee (0), Speed, and Max fee.

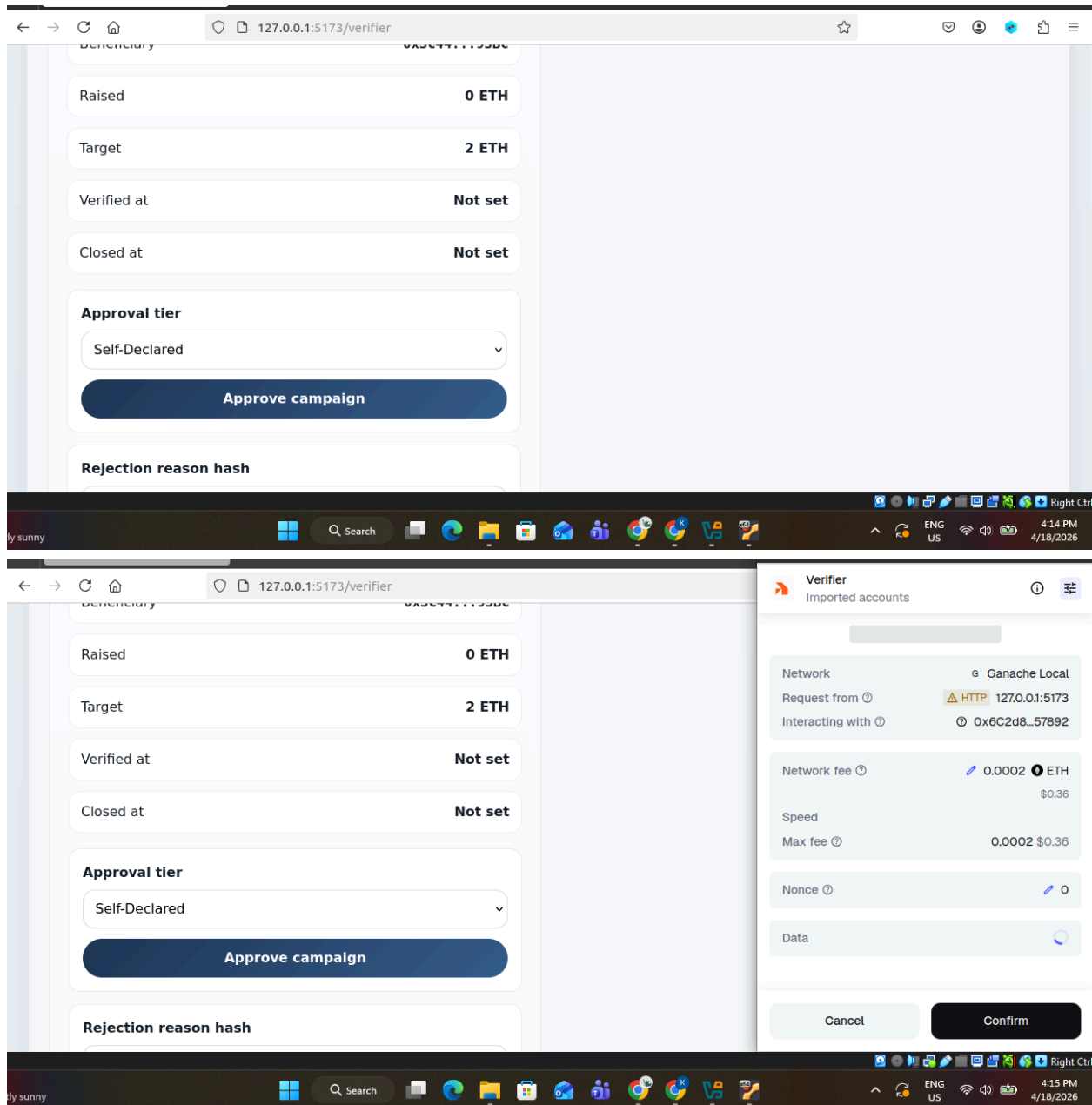
The screenshot displays the Charity Donation Platform interface. A transaction request overlay from MetaMask is visible on the right side of the screen. The overlay shows details for a transaction on the Ganache Local network, including the request from 127.0.0.1:5173, the interacting with address 0x6C2d8...57892, and the method setVerifier. The network fee is 0.0001 ETH (\$0.23), and the speed is 0.0001 \$0.23. The nonce is 151. The data is setVerifier. The overlay has 'Cancel' and 'Confirm' buttons.

The main interface shows the 'Verifier management' section, which includes a 'Refresh panel' button, a 'Verifier wallet' field with the address 0x70997970C51812dc3A010C7d01b50e0d17dc79C8, and a 'Permission' dropdown menu set to 'Grant verifier access'. A green bar indicates 'Waiting for MetaMask confirmation...'. Below this, there is a 'Lifecycle state snapshot' section.

The 'Verifier and admin control panel' section shows the current role as 'verifier' and the verifier access as 'Granted'. It also displays 'CAMPAIGNS LOADED' as 1 and 'ACTIONABLE VIEW' as 1. A row of buttons includes 'Actionable', 'All', 'Pending', 'Active', 'Paused', 'Completed', and 'Rejected'. A 'Refresh panel' button is also present.

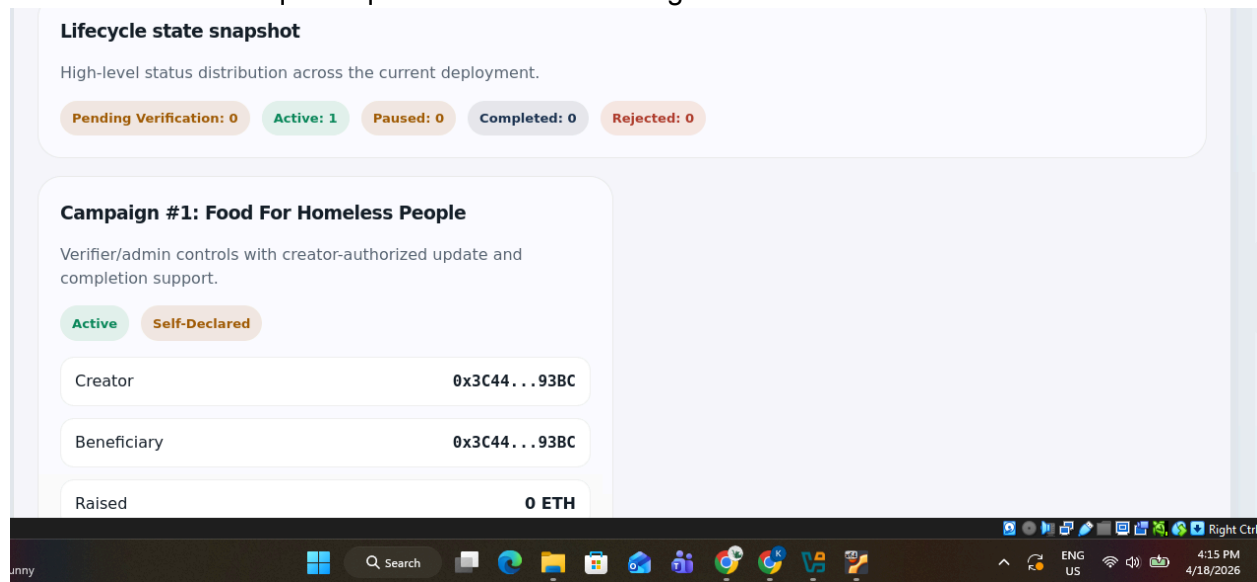
The 'Role trust summary' section shows visible moderation signals for the connected wallet. It lists 'Owner-admin' with a 'No' status, 'Verifier' with a 'Yes' status, 'Pending campaigns' with a count of 1, and 'Paused campaigns' with a count of 0.

The 'Lifecycle state snapshot' section is visible at the bottom of the interface.



A new campaign transaction is available for you to sign using MetaMask. Once you have signed the file, your campaign will be recorded as pending verification until an administrator verifies your campaign. The event called CampaignCreated will fire, and each campaign will be

numbered with a unique sequential identifier starting from 1.



**● NETWORK STATUS**  
Connected to the expected Ganache network. Contract reads are enabled.  
**Chain ID 1337**

**Verifier and admin control panel**  
Phase 9 exposes lifecycle moderation, proof publishing, and completion controls.

**● CURRENT ROLE**  
**donor**

**● VERIFIER ACCESS**  
**Not granted**

**● CAMPAIGNS LOADED**  
**1**

**● ACTIONABLE VIEW**  
**0**

Actionable

All

Pending

Active

Paused

Completed

Rejected

**Refresh panel**

**Role trust summary**  
Visible moderation signals for the connected wallet

Owner-admin

No

Verifier

No

Pending campaigns

0

Paused campaigns

0

**Food For Homeless People**  
Contract-backed campaign detail for Phase 7.

Active

Self-Declared

Category

Food

Raised

0 ETH

Target

2 ETH

Creator

0x3C44...93BC

Beneficiary

0x3C44...93BC

Distributing the food

**Donate to this campaign**  
Phase 8 adds the donor transaction flow while keeping the trust indicators visible.

**● DONATION COUNT**  
**0**

**● FUNDING PROGRESS**  
**0%**

**● PROOF ENTRIES**  
**0**

**Donation amount (ETH)**  
0.10



Creator

0x3C44...93BC

Beneficiary

0x3C44...93BC

Description hash

Distribuitng the food everywhere

Latest proof hash

0x00000000000000000000000000000000  
00000000000000000000000000000000  
0000000000

Created

4/18/2026, 6:41:11 AM

Verified

4/18/2026, 6:45:16 AM

Closed

Not set

PROOF ENTRIES

0

Donation amount (ETH)

0.10

Donations are recorded on-chain and forwarded directly to  
0x3C44CdDd86a900fa2b585dd299e03d12FA4293BC.

Donate with MetaMask

Open donor dashboard

Refresh detail

Donor dashboard

← → ↺ 🏠 🔍 Search 📁 🌐 🗂️ 📅 💬 👤 🔄 ⚙️ 🛡️ 🧰

🔒 127.0.0.1:5173/campaigns/1

Raised

0 ETH

Target

2 ETH

Creator

0x3C44...93BC

Beneficiary

0x3C44...93BC

Description hash

Distribuitng the food everywhere

Latest proof hash

0x00000000000000000000000000000000  
00000000000000000000000000000000  
0000000000

Created

4/18/2026, 6:41:11 AM

Donor Imported accounts

Transaction request

NetworkGanache Local

Request fromⓘ⚠ HTTP 127.0.0.1:5173

Interacting with ⓘ0x6C2d8...57892

Method ⓘDonate To Campaign

Amount

0.67 ETH

Network fee ⓘ

✎ 0.0009 ⚫ ETH \$2.04

Speed

Max fee ⓘ0.0009 \$2.04

Nonce ⓘ

✎ 0

Cancel

Confirm

The screenshot displays the user interface of a Blockchain-Based Charity Donation Platform. The interface is divided into several sections:

- Active Campaigns:** A list of active campaigns with filters for 'Active' and 'Self-Declared'. The selected campaign is 'Food'.
- Campaign Details:**
  - Category:** Food
  - Raised:** 0.67 ETH
  - Target:** 2 ETH
  - Creator:** 0x3C44...93BC
  - Beneficiary:** 0x3C44...93BC
  - Description hash:** Distributing the food everywhere
  - Latest proof hash:** 0x0000000000000000000000000000000000000000000000000000000000000000
- Summary Cards:**
  - DONATION COUNT:** 1
  - FUNDING PROGRESS:** 34%
  - PROOF ENTRIES:** 0
- Donation amount (ETH):** 0.10
- Donations are recorded on-chain and forwarded directly to:** 0x3C44CdDd86a900fa2b585dd299e03d12FA4293BC.
- Donation history snapshot:**
  - 0.67 ETH** (4/18/2026, 6:47:32 AM)
  - Donor: 0x90F7...b906

The interface also includes a 'Donor dashboard' button and a 'Refresh detail' button. A notification states: 'Latest donation receipt captured. Review the donor dashboard for the full donor-side view.'

## 11.3 Donation with Impact Receipt

A donation transaction sends ETH from a donor's MetaMask wallet directly to the recipient's wallet without requiring an intermediate balance in the contract. The ETH donor transfers ETH directly to the recipient's wallet via a single atomic transaction. The transaction receipt contains the transaction hash and can be verified via the Ganache block explorer independently from the donor's Ethereum address.

### Donor impact receipts

Each entry is reconstructed from immutable on-chain donation history.

● RECEIPT #1

0.67 ETH

**Food For Homeless People**

Active Self-Declared

Donated at4/18/2026, 6:47:32 AM

Beneficiary0x3C44...93BC

Donor0x90F7...b906

View campaign detail

### Support footprint

Quick trust-facing summary of the campaigns this wallet has backed.

Food For Homeless PeopleActive

Phase 8 adds donation tracking, donor impact receipts, and campaign support visibility.

● TOTAL DONATED

0.67 ETH

● RECEIPTS

1

● SUPPORTED CAMPAIGNS

1

● VERIFIED SUPPORTED

1

Refresh donor history

Browse campaigns

### Donor impact receipts

Each entry is reconstructed from immutable on-chain donation history.

● RECEIPT #1

0.67 ETH

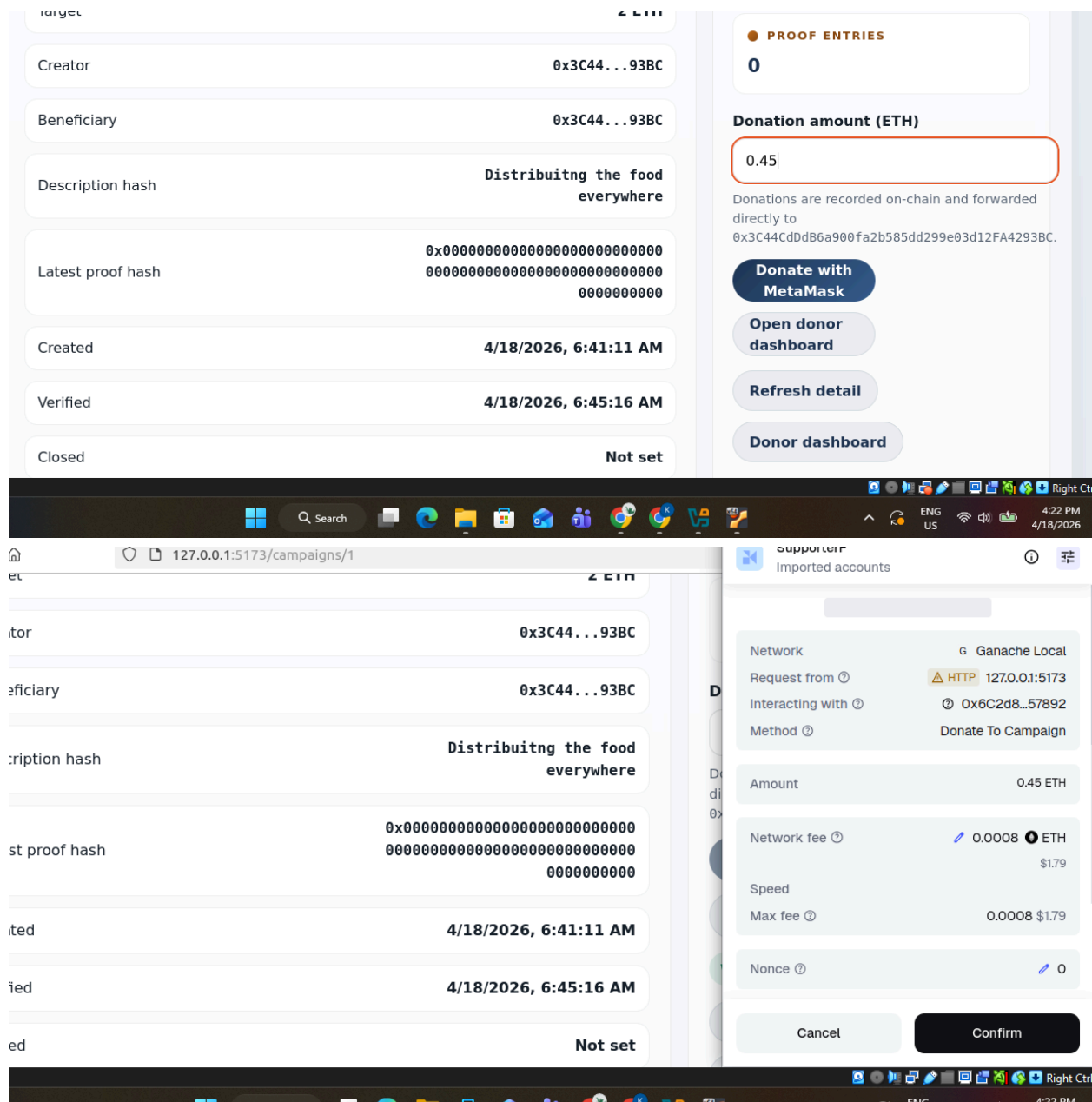
**Food For Homeless People**

Active Self-Declared

### Support footprint

Quick trust-facing summary of the campaigns this wallet has backed.

Food For Homeless PeopleActive



## 11.4 Donor Dashboard

The donor dashboard reads `getDonationsByDonor(wallet.account)` from the contract, providing a complete immutable record of all charitable contributions made by the connected wallet across all campaigns.

The verifier/creator of matter posts every proof update via the platform to a blockchain. These proof updates will be permanently recorded on the blockchain, and the updates can be inspected in the campaign detail view organized in order of when each update occurred. Through this information, you will be able to know exactly how the donation was used.

Phase 8 adds donation tracking, donor impact receipts, and campaign support visibility.

**TOTAL DONATED**  
**0.45 ETH**

**RECEIPTS**  
**1**

**SUPPORTED CAMPAIGNS**  
**1**

**VERIFIED SUPPORTED**  
**1**

[Refresh donor history](#)[Browse campaigns](#)

Current wallet context

**Wallet** 0x02484cb50aac86eae85610d6f4bf026f30f6627d

**Network** Ganache ready

**Active supported** 1

**Donor impact receipts**

Each entry is reconstructed from immutable on-chain donation history.

**RECEIPT #2**

**Food For Homeless People**

**0.45 ETH**

[Active](#)[Self-Declared](#)

**Support footprint**

Quick trust-facing summary of the campaigns this wallet has backed.

[Food For Homeless People](#) [Active](#)

Closed

Not set

Donor dashboard

**Recent proof and status updates**

Timeline rendering becomes richer later, but the data is already visible now.

No proof updates have been published for this campaign yet.

**Donation history snapshot**

The full donor dashboard and donation actions arrive in later phases.

**0.67 ETH**

4/18/2026, 6:47:32 AM

Donor: 0x90F7...b906

**0.45 ETH**

4/18/2026, 6:52:39 AM

Donor: 0x0248...627D

## 12. Security Analysis

---

### 12.1 Reentrancy Attack Prevention

The `donateToCampaign()` function is the only function which can transfer ETH. It has the OpenZeppelin's `nonReentrant` modifier, which places a lock on the function during execution and frees the lock after execution; any attempt to call `donateToCampaign` again (through a malicious recipient contract) during the ETH transfer will fail immediately and will revert the transaction. This directly prevents the class of attack known as "the DAO drain" which resulted in the loss of 60 million USD in 2016.

### 12.2 ETH Transfer Safety

The contract uses the preferred `.call{value:}()` function for sending funds, as opposed to either the old/legacy `.transfer()` and `.send()`. These two legacy methods can only send 2300 gas, which will not be enough to complete the transfer if the recipient has a more complex fallback function due to EIP-1884; `.call` forwards any and all available gas to the receiving contract. The calling contract will know for certain, because the `.call` is required to return a value when complete; if no value returned, the call will reverse using a user-defined (not standard) `revert`. Thus there can exist no 'silent' loss of funds.

### 12.3 Role-Based Access Control

Accessing functions with improper permissions will result in a custom error (`Unauthorized`) being returned. Additionally, access verification for those users performing verification can be easily tracked using a mapping (`mapping(address => bool)`).

### 12.4 Input Validation

At each function's beginning, input validation checks are performed using custom error messages. The title, descriptionHash, and category (string inputs) will be verified to contain non-empty byte arrays. The beneficiaryWallet will use `address(0)` as an invalid address for validation. The TargetAmount must be greater than zero. The ProofHash must also be greater than zero if the update occurs. These rules ensure that no malformed campaigns enter the system.

### 12.5 Zero Custody Model

Donor funds are never held within the contract. Donations are made by transferring ETH from `msg.sender` to `campaign.beneficiaryWallet`, all as part of one atomic transaction, which also records the donation. If any part of this transaction fails, then everything associated with that transaction is also rolled back, including the donation record itself. Therefore, there are no situations in which the contract doesn't send ETH after getting it from a donor.

### 12.6 Immutability Guarantees

All campaign records, donation records, and proof updates are stored in contract storage on the Ganache blockchain. Once mined, these records cannot be altered, deleted, or censored — providing the tamper-proof audit trail required by the PRD.

| Threat                     | Mitigation                  | Implementation                                     |
|----------------------------|-----------------------------|----------------------------------------------------|
| Reentrancy attack          | nonReentrant modifier       | OpenZeppelin ReentrancyGuard on donateToCampaign() |
| Unauthorized admin actions | onlyOwner modifier          | OpenZeppelin Ownable for setVerifier()             |
| Fraudulent campaigns       | Verification gate           | Donations blocked until verifier approves          |
| Silent ETH loss            | Return value check          | .call return checked, reverts with custom error    |
| Integer overflow           | Solidity ^0.8.x             | Built-in overflow protection since Solidity 0.8.0  |
| Duplicate verifier IDs     | mapping(address => bool)    | Set semantics — adding same address is idempotent  |
| Malformed inputs           | Custom error validation     | 13 custom errors covering all input edge cases     |
| Front-running donations    | N/A (ordering not critical) | Donation order does not affect fund distribution   |

## 13. GitHub Repository Link

The complete source code, including smart contracts, Truffle configuration, migration scripts, test suite, and React frontend, is available at the following repository:

[https://github.com/ProSai45/Charity\\_Blockchain](https://github.com/ProSai45/Charity_Blockchain)

Repository structure:

- contracts/ — CharityDonationPlatform.sol and Migrations.sol
- migrations/ — Truffle deployment scripts (1\_initial\_migration.js, 2\_deploy\_charity.js)
- test/ — 46 JavaScript test files covering all contract functions
- client/ — React + Vite frontend application
- client/src/abi/ — Truffle artifact bridge for auto contract address resolution
- docs/ — Implementation plan, architecture, test plan, demo guide
- truffle-config.js — Compiler and network configuration
- package.json — Root scripts: compile, test, migrate:reset, client:dev



## 14. Conclusion

---

By using Blockchain Technology to run an immutable Solidity Smart Contract which serves as an application logic for Donor, Recipient and Charity Organization for Charity Donations via an Ethereum compatible Blockchain, the Blockchain Based Charity Donation Platform illustrates how Decentralized Technology has the ability to fundamentally change the Trust Model of Charitable Donations. The Charity Donation Platform removes the need for intermediaries to hold funds prior to being donated; provides real time transparency in tracking the funds being donated and also provides an Auditable Permanently Stored Record of All Campaign Activity that occurs on the Platform.

The major accomplishments of this Project are:

- With an architecture that relies on "Zero-Custody" donations, ETH funds flow directly from the donor to their chosen beneficiary through a single atomic transaction. In other words, the contract will not hold any funds so there is no risk of misappropriation through fraud.
- The second layer of prevention from fraudulent activity is implemented by having a tiered verification process before any campaign can accept donations. A campaign must be approved by a recognized verifier as one of three types (Self-Declared, NGO-Verified, or Admin-Verified) in order to receive donations.
- A third layer of prevention is the immutable proof trail that does exist. Every proof that has been recorded for every campaign, from the original campaign documents to the closure proofs, will be permanently stored on-chain and provide verifiable evidence to donors as to how their funds were utilized.
- Non-Production-ready Architecture: A comprehensive set of automated tests (46) has been created and run against the contract. These tests validate every single function, edge case and access control rule to ensure the contract behaves as intended prior to any real deployment.
- Production-ready Architecture: The underlying technology stack (Solidity 0.8.20, Truffle, Ganache, React, ethers.js v6, MetaMask) is identical to that which would be utilized for a public testnet or mainnet deployment, so scaling can occur with a simple change in the network configuration.

Future improvements may involve the use of IPFS as a decentralized document storage method, deployment of the solution to the Ethereum Sepolia test network (which would allow for public access), establishing multi-signature controls over governance related to campaign approval, providing donor rewards through tokens, and creating mobile wallets that work with the WalletConnect protocol.

The project shows that blockchain technology provides more than just theory; it provides a powerful solution to real-life issues (i.e., transparency and accountability) regarding charitable giving

## 15. References

---

1. Ethereum Foundation. (2024). Ethereum Documentation. <https://ethereum.org/en/developers/docs/>
2. OpenZeppelin. (2024). OpenZeppelin Contracts Documentation — Ownable and ReentrancyGuard. <https://docs.openzeppelin.com/contracts/5.x/>
3. Truffle Suite. (2024). Truffle Framework Documentation. <https://trufflesuite.com/docs/truffle/>
4. Ganache Documentation. (2024). Ganache CLI — Personal Ethereum Blockchain. <https://trufflesuite.com/docs/ganache/>
5. ethers.js. (2024). ethers.js v6 Documentation. <https://docs.ethers.org/v6/>
6. MetaMask. (2024). MetaMask Developer Documentation. <https://docs.metamask.io/>
7. Solidity Team. (2024). Solidity Documentation v0.8.20. <https://docs.soliditylang.org/en/v0.8.20/>
8. Buterin, V. (2013). Ethereum White Paper. <https://ethereum.org/en/whitepaper/>
9. Vitejs. (2024). Vite — Next Generation Frontend Tooling. <https://vitejs.dev/>
10. React Team. (2024). React Documentation. <https://react.dev/>
11. Wood, G. (2014). Ethereum Yellow Paper — A Formal Specification of Ethereum. <https://ethereum.github.io/yellowpaper/>
12. ConsenSys. (2020). Ethereum Smart Contract Best Practices — Security. <https://consensys.github.io/smart-contract-best-practices/>